**VILNIUS UNIVERSITY OF APPLIED SCIENCES**

**FACULTY OF ELECTRONICS AND INFORMATICS**

# REPORT

# Academic Information System

| | |
|---|---|
| **Student:** | Abdelmoudjib Messai<br>Djelloul Ahmed ouail |
| **Project:** | Academic-system |
| **Supervisor:** | Dr. V. Liubinas |
| **Date:** | January 9, 2026 |

## TABLE OF CONTENTS

LIST OF ABBREVIATIONS ..... II
LIST OF FIGURES ..... III
LIST OF TABLES ..... IV
1. INTRODUCTION ..... 1
1.1 Relevance of the Project ..... 1
1.2 Project Objective ..... 1
1.3 Tasks ..... 1
1.4 Research Methods ..... 2
2. TECHNICAL DOCUMENTATION ..... 3
2.1 System Architecture ..... 3
2.2 Class Diagram ..... 5
2.3 Database Structure (ERD) ..... 7

2.4 Methods Documentation ..... 9
3. USER DOCUMENTATION (USER MANUAL) ..... 11
3.1 Task Formulation ..... 11
3.2 System Requirements ..... 12
3.3 Installation and Setup ..... 13
3.4 Login Data Generation ..... 14
3.5 User Roles and Functions ..... 15
3.6 Common Use Cases and Troubleshooting ..... 17
4. CONCLUSIONS ..... 19
5. REFERENCES ..... 20

## LIST OF ABBREVIATIONS

| | |
|---|---|
| **ERD** | Entity-Relationship Diagram |
| **CRUD** | Create, Read, Update, Delete |
| **RBAC** | Role-Based Access Control |
| **GUI** | Graphical User Interface |
| **BSON** | Binary JSON |
| **JDK** | Java Development Kit |
| **NoSQL** | Non-Relational Structured Query Language |
| **MVC** | Model-View-Controller |

## LIST OF FIGURES

Figure 1. Class Relationship Diagram with Directed Associations ..... 6
Figure 2. Entity Relationship Diagram (ERD) Mapping ..... 8

# LIST OF TABLES

Table 1. Project Technology Stack ..... 7
Table 2. Collection Definitions and Data Types ..... 9
Table 3. Troubleshooting and Error Resolution Matrix ..... 11

# 1. INTRODUCTION

## 1.1 Relevance of the Project

In the contemporary educational landscape, the move towards digitalization is no longer optional. Institutions like the Vilnius University of Applied Sciences (VUAS) require high-performance systems to manage the complex interplay between students, academic staff, and administrative cohorts. Traditional paper-based or disjointed spreadsheet systems are prone to human error, data redundancy, and security vulnerabilities. This project, entitled **"Academic Information System"**, addresses these critical needs by providing a centralized, secure, and intuitive platform for managing the lifecycles of students and academic courses.

## 1.2 Project Objective

The primary objective is to design and develop a robust, scalable, and multi-user desktop application that automates essential academic workflows. This includes the management of user identities (Administrators, Teachers, and Students), the organization of academic cohorts (Groups), the scheduling of educational modules (Subjects), and the systematic recording and retrieval of student grades.

## 1.3 Tasks

1. Analyze the requirements of an academic environment to identify core entities and their complex relationships.
2. Design a polymorphic object-oriented model in Java that allows for flexible user role management and extensible system features.
3. Implement a high-performance persistence layer using MongoDB, utilizing its document-oriented nature to store varied user profiles.
4. Develop a multi-view Graphical User Interface (GUI) using Java Swing, tailored to the specific functional needs of different user roles.
5. Ensure system stability through thorough testing of database connections and service-level business logic.

## 1.4 Research Methods

- **Systematic Analysis**: Evaluating common bottlenecks in academic administration to derive software requirements.
- **Object-Oriented Design (OOD)**: Applying design patterns (such as Repository and Service patterns) to ensure code maintainability.
- **NoSQL Database Modeling**: Researching efficient ways to model many-to-many relationships in a document-based database.
- **Iterative Prototyping**: Developing and refining Swing components to optimize user experience and form validation.

# 2. TECHNICAL DOCUMENTATION

## 2.1 System Architecture

### Architectural Layers

The system utilizes a four-tier architecture to ensure strict separation of concerns:

1. **Presentation Layer (UI)**: Utilizes Java Swing to provide a native desktop experience. Each user role is served by a dedicated dashboard class ensuring role-based functionality.
2. **Service Layer**: The `AcademicService` class acts as the central brain, orchestrating data flow and applying business rules.

3. **Data Access Layer (Repository)**: Implements interface-based repositories (e.g., `IUserRepository`) to handle BSON serialization and MongoDB interactions.
4. **Database Layer**: A MongoDB Community Server instance provides the storage engine for JSON-like documents.

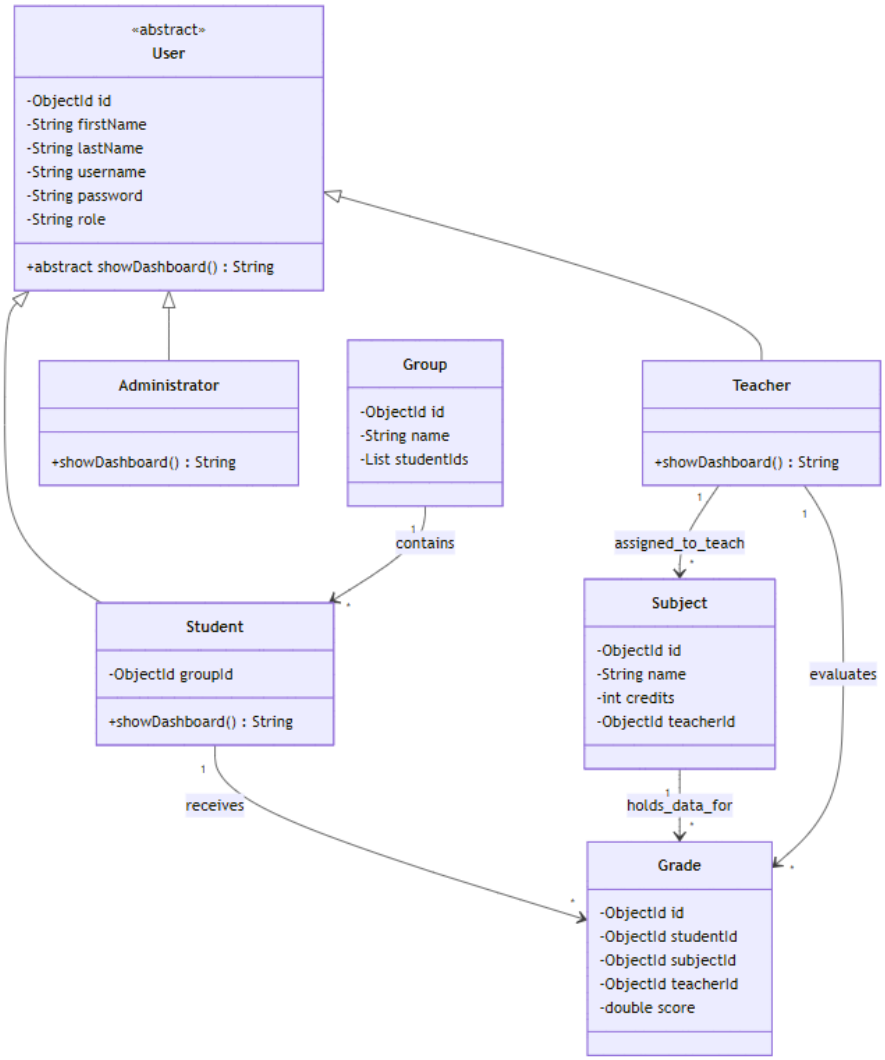
**Table 1. Project Technology Stack**

| Component | Technology | Version / Details |
|---|---|---|
| Runtime | Java Development Kit (JDK) | 17 (LTS) |
| Database | MongoDB Community Server | 4.4+ |
| GUI Framework | Java Swing | Standard Java Library |
| Build Tool | Apache Maven | Project Object Model (pom.xml) |
| NoSQL Driver | MongoDB Java Driver | 4.11.1 (Sync) |

## 2.2 Class Diagram

The core of the system revolves around the `User` abstract class, which is extended by `Student`, `Teacher`, and `Administrator`. This uses polymorphism for role handling. Academic entities like `Subject`, `Grade`, and `Group` are linked via ObjectIds. Arrow types denote: (----|>) for Inheritance and (---->) for directed associations.

**Figure 1. Class Relationship Diagram with Directed Associations**



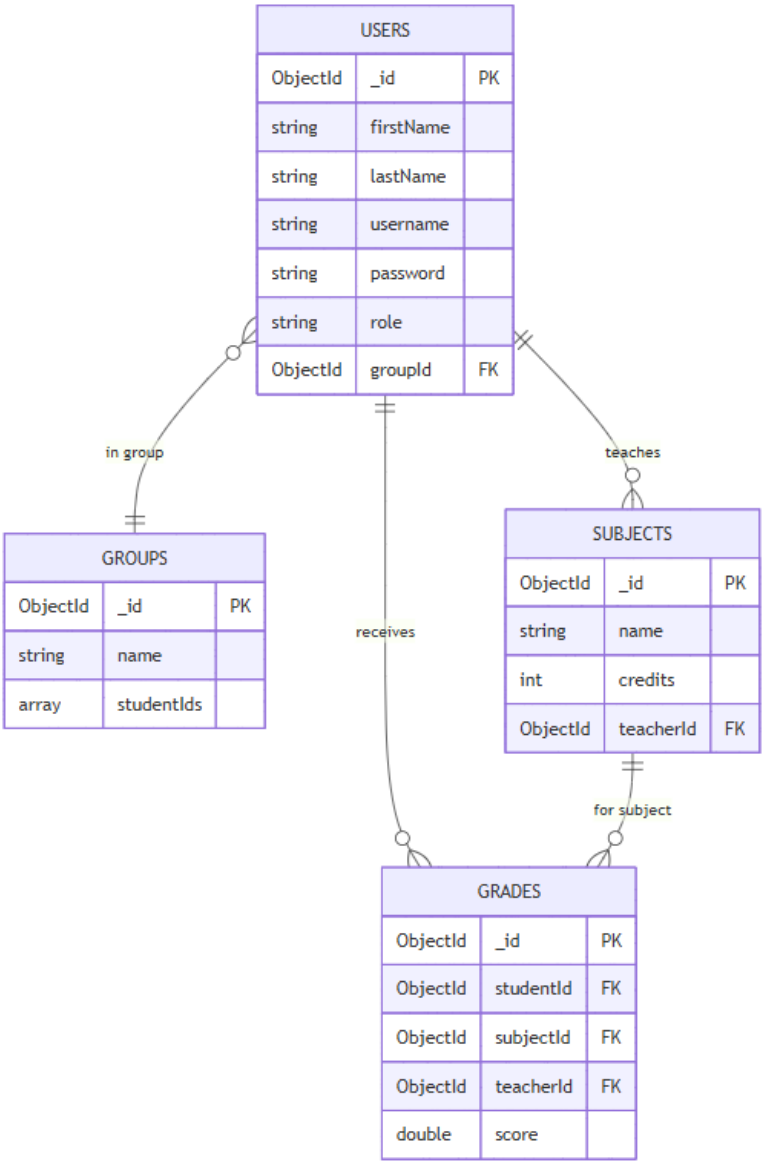
## 2.3 Database Structure (ERD)

The database uses **ObjectIds** for all primary keys (`_id`), which are 12-byte identifiers that guarantee uniqueness. Relationships are maintained via "Manual References" storing the `_id` of the related document.

**Table 2. Collection Definitions and Data Types**

| Collection | Field | Type | Description |
|---|---|---|---|
| **users** | firstName/lastName | String | Identifying info |
| | role | String | ADMIN, TEACHER, STUDENT |
| | groupId | ObjectId | Present for Students |
| **groups** | name | String | Human-readable name |
| | studentIds | Array[ObjectId] | List of member students |

**Figure 2. Entity Relationship Diagram (ERD) Mapping**




## 2.4 Methods Documentation

### Class: AcademicService

- **`User login(String username, String password)`**: Validates credentials by performing a lookup in the `users` collection. It returns the polymorphic `User` instance to the UI layer.

- **`void enterGrade(ObjectId studentId, ObjectId subjectId, ObjectId teacherId, double score)`**: This is a critical "upsert" method. It queries if a grade exists for the student/subject pair. If found, it updates the score; otherwise, it persists a new `Grade` document.
- **`List<Grade> getGradesForStudent(ObjectId studentId)`**: Retrieves all grade documents linked to the student for profile presentation.

# 3. USER DOCUMENTATION (USER MANUAL)

## 3.1 Task Formulation

The **Academic System** provides a secure portal for three distinct types of users:

- **Administrators**: System architects who define the "world" (Users, Groups, Subjects).
- **Teachers**: Functional users who manage the specific student performance metrics.
- **Students**: Passive consumers of data who monitor their own progress.

## 3.2 System Requirements

- **Hardware**: CPU 2.0GHz+, 2GB RAM+, 200MB Disk+.
- **Software**: Java Runtime Environment 17+, MongoDB Server 4.4+.

## 3.3 Installation and Setup

1. Start the MongoDB `mongod` service on port 27017.
2. Extract the application binaries.
3. Execute the jar: `java -jar academic-system.jar`.

## 3.4 Login Data Generation

- **Bootstrap Admin**: User `Admin` / Pass `Admin`.
- **Account Policy**: Administrators create accounts via the GUI. Usernames default to the first name; passwords default to the last name.

## 3.5 User Roles and Functions

- **Admin**: Full access to User creation, Group definitions, and Subject assignments.
- **Teacher**: Dropdown selection of assigned subjects; streamlined grading table.
- **Student**: Real-time updated table of individual subject scores and total credits.

## 3.6 Common Use Cases and Troubleshooting

**Table 3. Troubleshooting and Error Resolution Matrix**

| **Issue** | **Probable Cause** | **Solution** |
|---|---|---|
| "Connection Error" | MongoDB not started | Run mongod service. |
| "Invalid Credentials" | Case sensitivity | Check Caps Lock. |
| Table is empty | No subjects assigned | Contact Admin for assignment. |

# 4. CONCLUSIONS

The implementation of the Academic Information System successfully fulfills the objective of providing a centralized digital solution for educational data management. Throughout the development lifecycle, the focus remained on creating a system that is not only functional but also architecturally sound and user-friendly.

## 4.1 Project Achievements

**Operational Efficiency**: The system automates key processes such as user registration, subject assignment, and grade recording, significantly reducing the time required for administrative tasks compared to manual paper-based methods.
**Technical Robustness:** By utilizing Java 17 and the MongoDB sync driver, the application maintains high performance and data integrity. The choice of a NoSQL database allowed for the efficient storage of polymorphic user data (varying fields for Students vs. Teachers) within a single collection.
**Security and Access Control:** The implementation of Role-Based Access Control (RBAC) ensures that sensitive academic data is only accessible to authorized personnel, protecting student privacy while empowering teachers and administrators.

## 4.2 Lessons Learned

This project provided deep insights into the benefits of Object-Oriented Design (OOD) patterns, specifically the Repository and Service patterns. These allowed for a clean separation of business logic from database operations. Additionally, the project highlighted the flexibility of NoSQL schemas, demonstrating how document-oriented storage can evolve alongside changing academic requirements without the need for complex migrations.

## Future Outlook

While the current version serves as a fully functional prototype, there is significant potential for expansion. Future iterations could include:
**Security Hardening:** Integrating BCrypt encryption for password storage.
**Reporting Modules:** Developing a PDF generation feature to export student transcripts and academic reports.
**Cloud Deployment:** Moving the local MongoDB instance to MongoDB Atlas to enable multi-location access for larger institutions.

# 5. REFERENCES

1. Oracle. (2023). Java SE 17 Platform.
2. MongoDB, Inc. (2023). Java Driver 4.11.
3. Sommerville, I. (2015). Software Engineering (10th ed.).
4. Bloch, J. (2018). Effective Java (3rd ed.).